PARTE 1 · TU IDEA EN UNA FRASE

Antes de lo técnico, lo humano. Si no puedes responder esto, lo técnico no importa. **1.1**

Completa esta frase en **una sola oración**, sin palabras técnicas:

Mi aplicación sirve para que los jóvenes y las personas curiosas puedan descubrir contenidos que conecten sus pasiones con la ciencia, la cultura y el conocimiento, sin tener que buscar información en diferentes plataformas ni perder tiempo en contenidos superficiales.

> 💡 Ejemplo: *"Mi aplicación sirve para que los dueños de mascotas puedan reservar un paseo sin tener que llamar por teléfono."*

1.2 ¿Cómo hace **hoy** la gente eso que tu app va a resolver? ¿Por WhatsApp? ¿En un cuaderno? ¿No lo hace?

Actualmente, las personas utilizan plataformas como TikTok, Instagram, YouTube y Netflix para encontrar contenido relacionado con sus intereses. Sin embargo, la información educativa suele estar dispersa y no siempre es fácil encontrar contenidos que conecten el entretenimiento con el aprendizaje.

1.3 Si mañana tu aplicación desapareciera, **¿quién se daría cuenta y por qué le dolería?**

Los jóvenes y usuarios interesados en aprender a través de sus pasiones, porque perderían un espacio donde pueden explorar sus gustos y relacionarlos con nuevos conocimientos científicos y culturales.

1.4 Nombra **una** aplicación que ya exista y se parezca a la tuya. ¿Qué le copiarías y qué harías distinto?

Le copiaría: a YouTube y Netflix, la facilidad para descubrir contenidos y explorar recomendaciones.

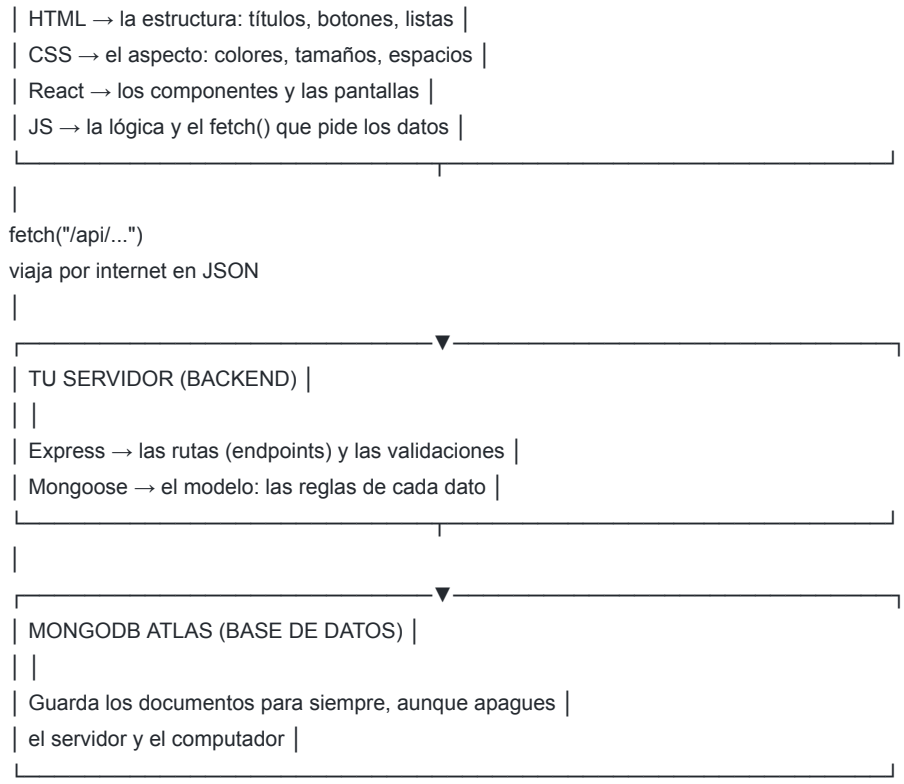
Haría distinto: ETHEOS conectaría directamente las pasiones de las personas con el conocimiento, promoviendo el aprendizaje y el pensamiento crítico en lugar del consumo pasivo de contenido.

> ⚠️ **Señal de alerta:** si en la 1.1 escribiste *"sirve para todo"* o *"para que la gente vea mi negocio"*, todavía no tienes un proyecto: tienes una intención. Vuelve y piensa en **una sola acción concreta** que alguien necesita hacer.

PARTE 2 · TU STACK: QUÉ HACE CADA TECNOLOGÍA

Tu aplicación se construye con seis tecnologías. Antes de diseñar nada, ten claro **qué hace cada una**, porque cada decisión que tomes en este taller le pertenece a una de ellas.





HTML + CSS	Navegador	Estructura y aspecto	Cuántas pantallas y cómo se ven
React	Navegador	Componentes reutilizables y pantallas	Qué componentes necesitas (Parte 8)
JavaScript	En los dos lados	La lógica de todo	—
Express	Servidor	Recibe peticiones y responde	Tus endpoints (Parte 7)
Mongoose	Servidor	Define cómo debe ser cada dato	Tus modelos (Parte 5)
MongoDB	Nube (Atlas)	Guarda la información	Tus colecciones (Parte 5)

2.1 Para comprobar que lo tienes claro, responde con una palabra cada una:

¿Quién pinta los botones en la pantalla?	React, junto con HTML y CSS.
¿Quién guarda la información para que no se pierda?	MongoDB.

¿Quién recibe la petición y decide qué responder?	Express.
¿Quién impide que se guarde un producto sin precio?	Mongoose.
¿Quién le pide los datos al servidor desde el navegador?	`fetch()`.

💡 Respuestas: React (con HTML y CSS) · MongoDB · Express · Mongoose · `fetch()`

PARTE 3 · ¿QUÉ DEBE HACER TU APLICACIÓN?

3.1 Haz una lista de **todo lo que alguien debería poder hacer** en tu aplicación. Cada una empieza con un **verbo**.

💡 Ejemplos: *ver los productos · buscar por nombre · registrarse · iniciar sesión · agregar al carrito · hacer un*

pedido · ver mis pedidos · dejar una reseña · publicar un producto · editar mi perfil

Escribe **mínimo 10**. Sí, diez. Después recortamos.

1-Explorar el catálogo de contenidos.
2-Buscar contenidos por nombre o palabra clave.
3-Filtrar contenidos por categorías.
4-Consultar la descripción de un contenido.
5-Descubrir nuevas pasiones e intereses.
6-Guardar contenidos favoritos.
7-Enviar sugerencias de nuevos contenidos.
8-Consultar el estado de carga y los mensajes del sistema.
9-Iniciar sesión como administrador.
10-Aprobar o rechazar sugerencias de contenidos.

3.2 Ahora clasifica cada una:

🟢 **Imprescindible**	Sin esto, la app no sirve para nada	**Máximo 5**
🟡 **Deseable**	Mejora la experiencia, pero se puede vivir sin ello	Las que quieras
🔵 **Soñado**	Me encantaría, pero no es para este semestre	Las que quieras

⚠️ **Regla dura:** si tienes más de 5 verdes, **no has decidido todavía**. Vuelve y bájalas a 5. Un proyecto que lo intenta todo no termina nada.

3.3 De tus 5 verdes, **¿cuál es la número uno?** La que, si solo pudieras programar una, sería esa.

Mi acción número uno es explorar el catálogo de ETHEOS y descubrir contenidos que conecten las pasiones de los usuarios con la ciencia y la cultura.

Imprescindibles — MVP

1. Explorar el catálogo.
2. Buscar y filtrar contenidos.
3. Consultar los detalles de un contenido.
4. Enviar sugerencias.
5. Revisar y aprobar o rechazar sugerencias como administrador.

Deseables

- Guardar contenidos favoritos.
- Mostrar recomendaciones según los intereses.
- Mejorar los mensajes de carga, error y ausencia de resultados.
- Adaptar la interfaz a celulares y computadores.

Soñadas

- Crear perfiles personalizados.
- Formar comunidades por intereses.
- Integrar un asistente de inteligencia artificial real.
- Ofrecer recomendaciones personalizadas más avanzadas.

💡 Esa acción es tu **MVP** (producto mínimo viable): lo primero que vas a construir completo, del navegador a la base de datos.

PARTE 4 · HISTORIAS DE USUARIO

Esta es la parte más importante del taller. Una **historia de usuario** es una forma de escribir lo que hace tu app **desde el punto de vista de quien la usa**, no desde el código.

El formato

Como [tipo de usuario],
quiero [hacer algo],
para [conseguir algún beneficio].

Un ejemplo bien escrito

Como dueño de una mascota,
quiero reservar un paseo eligiendo el día y la hora,
para no tener que llamar por teléfono y esperar a que me contesten.

Por qué importa el "para"

Mira la diferencia:

❌	✅
"Quiero un botón azul"	"**Como** cliente, **quiero** ver claramente el botón de pagar, **para** no perderme en la pantalla"
"Hacer un formulario"	"**Como** dueño, **quiero** registrar mi mascota, **para** que el paseador sepa de qué tamaño es"

"Usar MongoDB"	"**Como** cliente, **quiero** ver mis pedidos anteriores, **para** repetir el que más me gustó"

💡 La columna de la izquierda habla de **código**. La de la derecha habla de **personas**. El código sale solo cuando sabes para quién es.

Criterios de aceptación

Cada historia necesita saber **cuándo está terminada**. Eso se escribe así:

Sé que esta historia está lista cuando:
- El usuario puede elegir un servicio de la lista
- Puede elegir fecha y hora
- Si deja un campo vacío, ve un mensaje claro y no se guarda nada
- Al confirmar, ve el mensaje "Reserva confirmada"
- La reserva aparece después en "Mis reservas"

4.1 Escribe **una historia de usuario por cada acción** 🟢 **verde** de la Parte 3. Mínimo 5.

Historia 1. Explorar el catálogo

Como usuario de ETHEOS,

quiero explorar un catálogo organizado por categorías,

para encontrar contenidos relacionados con mis intereses y descubrir nuevos conocimientos.

Sé que está lista cuando:

- Se muestran los contenidos disponibles en tarjetas organizadas.
- Cada tarjeta presenta información básica del contenido.
- Si no existen contenidos, aparece un mensaje informativo.
- Si ocurre un error al cargar los datos, se muestra un mensaje y una opción para intentar nuevamente.

Historia 2. Buscar y filtrar contenidos

Como usuario de ETHEOS,

quiero buscar contenidos por palabras clave y filtrarlos por categorías,

para encontrar rápidamente información relacionada con mis pasiones.

Sé que está lista cuando:

- Puedo escribir una palabra para buscar contenidos.
- Puedo seleccionar una categoría.
- Puedo limpiar la búsqueda y los filtros.
- Si no hay coincidencias, aparece un mensaje que lo explica.

Historia 3. Consultar los detalles de un contenido

Como usuario de ETHEOS,

quiero consultar la información detallada de cada contenido,

para comprender qué ofrece y cómo se relaciona con mis intereses y el conocimiento.

Sé que está lista cuando:

- Puedo abrir un contenido desde el catálogo.
- Se muestra su título, categoría y descripción.
- Se presentan los enlaces disponibles, cuando existan.
- Si el contenido no existe, aparece un mensaje claro.

Historia 4. Enviar sugerencias

Como usuario de ETHEOS,

quiero enviar sugerencias de contenidos mediante un formulario,

para contribuir al crecimiento y mejoramiento de la plataforma.

Sé que está lista cuando:

- Puedo ingresar mi nombre o alias, el enlace del recurso y el motivo de la sugerencia.
- Los campos obligatorios se validan antes de enviar.
- Recibo una confirmación cuando la sugerencia se guarda correctamente.
- Si ocurre un error, recibo un mensaje que me permite intentarlo nuevamente.

Historia 5. Administrar sugerencias

Como administrador de ETHEOS,

quiero revisar, aprobar o rechazar las sugerencias enviadas,

para mantener organizado el catálogo y controlar qué contenidos se publican.

Sé que está lista cuando:

- Puedo ingresar al panel de administración mediante autenticación.
- Puedo consultar las sugerencias pendientes.
- Puedo aprobar o rechazar cada sugerencia.
- El sistema registra el estado de cada sugerencia.
- Una persona sin autorización no puede acceder al panel.

💡 Copia ese bloque tantas veces como historias tengas. En tu archivo arquitectura.md van todas.

Historia	¿Qué pasa si falta un dato?	¿Qué pasa si no hay internet?	¿Qué pasa si no hay nada que mostrar?
1. Explorar	Se omite la información opcional y se valida la obligatoria.	Se muestra un mensaje de error y la opción de reintentar.	Mostrar que todavía no hay contenidos disponibles.
2. Buscar y filtrar	Se permite buscar sin texto o sin seleccionar una categoría.	Mostrar un error y permitir reintentar.	Mostrar que no se encontraron coincidencias.

3. Consultar detalles	Se muestra la información disponible sin errores.	Ocultar el dato opcional o mostrar información alternativa	Informar que el contenido no existe o no está disponible.
4. Enviar sugerencias	No enviar el formulario y señalar los campos pendientes.	No confirmar el envío; permitir reintentar.	No aplica: se muestra el formulario vacío.
5. Administrar	Mostrar los datos que falten como pendientes de completar.	Informar que no se pudo cargar o guardar el cambio.	Mostrar que no hay sugerencias pendientes.

4.2 Para cada historia, piensa **qué pasa cuando algo sale mal**. Es lo que separa una app amateur de una profesional:

PARTE 5 · ¿QUIÉN USA TU APLICACIÓN? (los roles) **5.1** Lista

los **tipos de persona** que entran a tu app. No personas concretas: tipos.

Rol	¿Qué puede hacer que los demás NO?	¿Qué NO debe poder hacer nunca?
Visitante o usuario	**Explorar, buscar, filtrar, consultar contenidos, guardar favoritos localmente y enviar sugerencias.**	**Aprobar sugerencias, modificar el catálogo oficial o acceder al panel administrativo.**
Administrador	**Revisar sugerencias, aprobarlas, rechazarlas y gestionar el contenido publicado.**	**Permitir accesos no autorizados o exponer contraseñas y datos confidenciales.**

💡 Casi todos los proyectos tienen entre 2 y 3: *cliente / vendedor / administrador*, *estudiante / tutor / admin*, *dueño / paseador / admin*.

5.2 ☐ Pregunta incómoda a propósito: **¿de verdad necesitas que la gente se registre?** Sí / No, porque

No, por ahora, porque el catálogo es público, las sugerencias pueden enviarse sin crear una cuenta y los favoritos pueden guardarse localmente en el navegador. Solo el administrador necesita autenticarse para gestionar las sugerencias

☐ **Regla práctica:** necesitas registro **solo si** la app tiene que recordar algo que le **pertenece** a esa persona (sus pedidos, sus reservas, sus publicaciones). Si tu app solo muestra información igual para todo el mundo, **no necesitas registro**, y acabas de ahorrarte semanas de trabajo.

PARTE 6 · ¿QUÉ INFORMACIÓN GUARDA? (MongoDB + Mongoose)

6.1 Vuelve a tus historias de usuario y **subraya todos los sustantivos**.

Sustantivos principales de las historias

Contenidos, pasiones, categorías, sugerencias, usuarios, administradores, favoritos y estados de revisión.

☐ *"reservar un paseo"*, *"ver los productos"*, *"dejar una reseña"*. Esos sustantivos son, casi siempre, **tus**

colecciones.

6.2 Escribe tus colecciones. **En plural, minúscula, sin tildes ni espacios.**

Mis colecciones

1. `pasiones`
2. `sugerencias`
3. `administradores`

☐ Si te salieron más de 4, sospecha. Un proyecto de este tamaño rara vez necesita más de 3 o 4.

6.3 Para **cada** colección, llena esta tabla:

Colección: `pasiones`

Campo	Tipo de dato	¿Obligatorio?	Ejemplo
`titulo`	String	Sí	Anime y ciencia
`slug`	String	Sí	anime-y-ciencia
`categoria`	String	Sí	Cultura pop
`descripcion`	String	Sí	Contenido que conecta anime con conceptos científicos.
`imagenUrl`	String	No	URL de una imagen
`enlaces`	Array	No	Lista de recursos relacionados
`publicada`	Boolean	Sí	`true`

Colección: `sugerencias`

Campo	Tipo de dato	¿Obligatorio?	Ejemplo
`nombre`	String	Sí	Usuario ETHEOS
`enlace`	String	Sí	`https://ejemplo.com/recurso`
`motivo`	String	Sí	Relaciona arte y conocimiento.
`estado`	String	Sí	`pendiente`
`fechaCreacion`	Date	Sí	Fecha de envío
`fechaRevision`	Date	No	Fecha de revisión
`revisadoPor`	ObjectId o referencia	No	Identificador del administrador

Colección: `administradores`

Campo	Tipo de dato	¿Obligatorio?	Ejemplo
`nombre`	String	Sí	Administrador ETHEOS
`email`	String	Sí	`admin@etheos.com`
`passwordHash`	String	Sí	Contraseña almacenada mediante hash seguro
`rol`	String	Sí	`admin`
`activo`	Boolean	Sí	`true`

Tipos disponibles en Mongoose: **String** (texto) · **Number** (número) · **Boolean** (sí/no) · **Date** (fecha) · **Array** (lista) · **Object** (objeto embebido)

6.4 Así se ve tu tabla convertida en código. Esto es lo que vas a escribir después:

```
const productoSchema = new mongoose.Schema({
 nombre: {
 type: String,
 required: [true, "El nombre es obligatorio"],
 trim: true
 },
 precio: {
 type: Number,
 required: [true, "El precio es obligatorio"],
 min: [0, "El precio no puede ser negativo"]
 },
 disponible: {
 type: Boolean,
 default: true
 }
}, { timestamps: true });
```

Fíjate: la columna "¿Obligatorio?" de tu tabla se convierte en `required`. El "tipo de dato" se convierte en `type`. **Tu tabla ES el código**, solo que escrita en español.

Se define un esquema de Mongoose para cada colección. En él se especifican los tipos de datos, los campos obligatorios, las validaciones y los valores predeterminados. Por ejemplo, `titulo` debe ser obligatorio y `publicada` puede tener `false` como valor predeterminado

6.5 Tres preguntas de control sobre tus campos:

1. ¿Qué campo no puede faltar? En `pasiones`, el título, la categoría y la descripción; en `sugerencias`, el nombre o alias, el enlace y el motivo; en `administradores`, el correo, la contraseña protegida y el rol.

2. ¿Qué campo debe ser único? El `slug` de cada pasión y el correo electrónico de cada administrador.

3. ¿Qué campo puede tener un valor predeterminado? El campo `publicada` puede iniciar en `false` y el campo `estado` de una sugerencia puede iniciar en `pendiente`.

6.6 **Datos sensibles.** Responde con honestidad:

- ¿Guardas contraseñas? Sí, para el administrador, pero nunca deben guardarse en texto plano. Debe almacenarse un hash seguro.
- ¿Guardas datos que no te gustaría que se filtraran? Sí, pueden existir credenciales administrativas y datos de contacto. Deben protegerse y limitarse al mínimo necesario.
- ¿De verdad necesitas cada uno de esos datos? No necesito solicitar documentos, direcciones ni teléfonos de los visitantes. Para enviar una sugerencia bastan los datos mínimos del formulario.

La autenticación segura, el hash de contraseñas y la conexión real a la base de datos son tareas que deben implementarse antes de considerar completo este diseño.

PARTE 7 · ¿CÓMO SE RELACIONAN TUS DATOS? 7.1

¿Alguna colección **necesita saber** de otra?

Sí. Las sugerencias aprobadas pueden convertirse en contenidos de la colección `pasiones`. Además, cada sugerencia puede guardar una referencia al administrador que la revisó. Los favoritos no necesitan una colección por ahora, porque se guardan en el navegador.

- [ ] Un **pedido** necesita saber **quién** lo hizo y **qué** productos lleva.

7.2 Para cada relación, elige una de las dos:

Guardarlo adentro (embebido)	Cuando el dato **solo** pertenece a ese documento y siempre se consulta con él	`{ nombre: "Ana", direccion: { ciudad: "Armenia" } }`
Guardar una referencia	Cuando el dato **vive por su cuenta** y lo usan varios	`{ cliente: "ana@mail.com", total: 45000 }`

- [ ] **Para empezar:** la dirección de un usuario va **adentro** (es solo suya). El cliente de un pedido va como **referencia** (el cliente sigue existiendo aunque borres el pedido).

Dato embebido: los enlaces relacionados pueden guardarse dentro de cada pasión, porque pertenecen a ese contenido.
Referencia: `revisadoPor` puede apuntar al identificador del administrador que revisó la sugerencia, porque el administrador existe independientemente de ella.

7.3 Dibuja tus colecciones y las flechas entre ellas. A mano, en una servilleta, donde sea. **Hazlo.**

administradores

Identidad y permisos

Un administrador puede revisar varias sugerencias

=>

sugerencias

Enlace, motivo y estado de revisión

Las sugerencias aprobadas pueden convertirse en contenidos

=>

pasiones

Catálogo de contenidos publicados

PARTE 8 · TUS ENDPOINTS (Express)

Historia	Verbo	Ruta	¿Qué devuelve?	Códigos

Ver todos los contenidos	GET	`/api/pasiones`	Lista de contenidos publicados	200 / 500
Ver un contenido	GET	`/api/pasiones/:id`	Detalle del contenido	200 / 404 / 500
Enviar una sugerencia	POST	`/api/sugerencias`	Confirmación y sugerencia creada	201 / 400 / 500
Iniciar sesión como administrador	POST	`/api/admin/login`	Confirmación de autenticación	200 / 401 / 500
Ver sugerencias pendientes	GET	`/api/admin/sugerencias`	Lista de sugerencias pendientes	200 / 401 / 500
Aprobar o rechazar una sugerencia	PATCH	`/api/admin/sugerencias/:id/estado`	Sugerencia actualizada	200 / 400 / 401 / 404 / 500

8.1 Reglas que debes cumplir al llenarla:

- **Verbo correcto:** si trae información → `GET`. Si crea → `POST`. Si modifica → `PUT`. Si borra → `DELETE`. • **Rutas en plural y minúscula:** `/api/productos`, no `/api/Producto`.
- **El verbo no se repite en la ruta.** `GET /api/productos` ya dice que trae; `/api/obtenerProductos` es redundante.
- **Un endpoint por acción.** Si una fila hace dos cosas, pártela en dos.

8.2 Así se ve una fila de tu tabla convertida en código:

```
app.get("/api/productos", async (req, res) => {
 try {
 const productos = await Producto.find();
 res.json(productos);
 } catch (error) {
 res.status(500).json({ error: "Error al consultar los productos" });
 }
});
```

8.3 Preguntas de control:

¿Qué endpoint utilizarías para ver todos los contenidos? `GET /api/pasiones`

¿Qué endpoint utilizarías para crear una sugerencia? `POST /api/sugerencias`

¿Qué endpoint utilizarías para modificar el estado de una sugerencia? `PATCH /api/admin/sugerencias/:id/estado`

¿Qué pasa si el contenido solicitado no existe? El servidor responde con el código `404 Not Found` y un mensaje que explique que el contenido no fue encontrado.

¿Qué pasa si ocurre un error en el servidor? El servidor responde con el código `500 Internal Server Error`, sin mostrar información confidencial.

¿Qué endpoint necesita protección? Las rutas administrativas deben exigir autenticación y autorización. Las rutas públicas del catálogo no necesitan que el visitante inicie sesión.

Estas rutas son el diseño propuesto para ETHEOS; no significa que todas estén implementadas actualmente.

PARTE 9 · TUS PANTALLAS Y COMPONENTES (React)

En React, una aplicación se arma con **componentes**: pedacitos de interfaz que se reutilizan. Una **pantalla** es un componente grande hecho de componentes pequeños.

```
PANTALLA: Catálogo
├── Encabezado (se repite en todas las pantallas)
├── Buscador
├── ListaDeProductos
│   ├── TarjetaProducto ← el mismo componente, 20 veces
│   ├── TarjetaProducto
│   └── TarjetaProducto
└── PiePagina (se repite en todas las pantallas)
```

9.1 Lista tus pantallas. Mínimo 3, máximo 6 por ahora.

Pantallas

Pantalla	¿Quién la ve?	Endpoint	¿Qué se muestra si no hay datos?
Inicio	Todos	Ninguno obligatorio	Presentación de ETHEOS y acceso al catálogo
Catálogo	Todos	`GET /api/pasiones`	“Pronto tendremos nuevos contenidos disponibles”.
Detalle de contenido	Todos	`GET /api/pasiones/:id`	“No encontramos este contenido”.
Buzón de sugerencias	Todos	`POST /api/sugerencias`	Formulario vacío con instrucciones.

Panel de administración	Administrador autenticado	`GET /api/admin/sugerencias` y `PATCH /api/admin/sugerencias/:id/estado`	“No tienes sugerencias pendientes”.

☐ La última columna la olvida **todo el mundo**, y es la que hace que tu app se vea profesional el primer día, cuando todavía no hay ni un solo producto cargado. *"Aún no tienes pedidos. ¡Haz el primero!"* es infinitamente mejor que una pantalla en blanco.

9.2 ¿Qué componentes se **repiten** en varias pantallas? Esos son los que vale la pena construir primero. Componentes

Componentes reutilizables

1. `Encabezado`
2. `Navegacion`
3. `PiePagina`
4. `TarjetaPasion`
5. `ListaPasiones`
6. `Buscador`
7. `FiltrosCategorias`
8. `EstadoCarga`
9. `MensajeError`
10. `MensajeVacio`
11. `FormularioSugerencia`
12. `FilaSugerencia`

9.3 Así se ve una pantalla que pide datos a tu API:

¿Qué debe mostrar la aplicación mientras carga?

Un mensaje como “Cargando contenidos…”.

¿Qué debe mostrar si no hay contenidos?

Un mensaje como “Aún no hay contenidos disponibles. Vuelve pronto”.

¿Qué debe mostrar si falla la conexión?

Un mensaje como “No pudimos cargar los contenidos. Revisa tu conexión e inténtalo de nuevo”.

PARTE 10 · EL MAPA COMPLETO

Dibuja el recorrido de **tu historia número uno**, desde el clic hasta la pantalla:

La historia principal será explorar el catálogo de ETHEOS.

1. **El usuario hace: clic en la sección del catálogo.**
2. **React ejecuta: `fetch("/api/pasiones", { method: "GET" })`.**
3. **Llega al endpoint: `GET /api/pasiones`.**
4. **Express valida: los filtros de búsqueda y categoría, si fueron enviados, y los parámetros de la solicitud.**
5. **Mongoose / MongoDB ejecuta: una consulta para buscar las pasiones publicadas en la colección `pasiones`.**
6. **Responde con el código: `200` y una lista de contenidos en formato JSON.**
7. **React muestra en pantalla: las tarjetas de las pasiones disponibles, con sus títulos, categorías y descripciones.**

Si no hay contenidos, React muestra el estado vacío. Si falla la solicitud, muestra un mensaje de error y permite volver a intentarlo.

Este es el verificador de todo el taller. Si puedes llenar el recorrido completo **sin huecos**, tienes la arquitectura resuelta. Si te trabas en un renglón, **ahí está exactamente lo que te falta aprender o decidir**. Anótalo: eso es lo que vas a preguntar en la próxima sesión.

PARTE 11 · ALCANCE Y RIESGOS

Esto es lo que separa un proyecto terminado de uno abandonado.

11.1 Escribe **tres cosas que tu aplicación NO va a hacer**, explícitamente

.

1-No va a procesar pagos reales ni ofrecer suscripciones Premium funcionales durante esta primera etapa.

2-No va a tener una aplicación móvil nativa.

3-No va a tener un modo colectivo para la concentración absoluta.

Ejemplos: *"no va a procesar pagos reales"* · *"no va a tener app de celular"* · *"no va a enviar correos"*. Esto no es rendirse: es **definir el alcance**, y es lo que hacen todos los equipos profesionales.

11.2 ¿Cuál es la parte que **menos sabes hacer**? Nómbrala. Es la que te va a frenar, y mientras antes la nombres, antes te podemos ayudar.

La parte que más necesito aprender es la conexión entre el frontend y el backend, especialmente la creación de endpoints con Express, el almacenamiento de datos con MongoDB y Mongoose y la protección del acceso administrativo.

11.3 Si solo tuvieras **dos semanas** para mostrar algo que funcione de verdad, ¿qué construirías?

Construiría un catálogo conectado a una base de datos real, con búsqueda y filtros, consulta del

detalle de cada contenido, un formulario que guarde sugerencias y un panel administrativo protegido para aprobarlas o rechazarlas.

También probaría los mensajes de carga, error y ausencia de datos. Dejaría para después las comunidades, los perfiles personales y la inteligencia artificial real.

TU ENTREGABLE

Crea en tu repositorio un archivo llamado **arquitectura.md** con esta estructura. Puedes copiar y pegar esta plantilla:

Arquitectura de ETHEOS

1. Qué es

ETHEOS es una aplicación que permite a los jóvenes descubrir contenidos que conectan sus pasiones con el conocimiento científico y cultural, sin tener que buscar información en múltiples plataformas.

2. Qué hace

- **Imprescindible (MVP): explorar el catálogo, buscar y filtrar contenidos, consultar detalles, enviar sugerencias y gestionar su aprobación.**
- **Deseable: favoritos, mejores recomendaciones y una experiencia adaptable a diferentes dispositivos.**
- **Fuera de alcance por ahora: pagos reales, aplicación móvil nativa, mensajería privada y mentor con inteligencia artificial real.**

3. Historias de usuario

Se incluyen las cinco historias de usuario de la Parte 4, con sus respectivos criterios de aceptación.

4. Roles

- **Visitante: consulta contenidos y envía sugerencias.**
- **Administrador: revisa sugerencias y gestiona los contenidos publicados.**

5. Modelo de datos

Colecciones `pasiones`, `sugerencias` y `administradores`, con los campos y tipos definidos en la Parte 6.

6. Endpoints

Se incluyen las seis rutas propuestas en la Parte 8, con sus verbos, respuestas y códigos de estado.

7. Pantallas y componentes

Inicio, catálogo, detalle, buzón de sugerencias y panel administrativo, con los componentes reutilizables descritos en la Parte 9.

8. Recorrido de la historia principal

El usuario abre el catálogo, React solicita los datos al endpoint de Express, Mongoose consulta MongoDB y el servidor devuelve los contenidos publicados para que React los muestre.

9. Decisiones y pendientes

- **Decidí mantener el catálogo público porque el conocimiento debe ser accesible sin necesidad de registrarse.**
- **Decidí utilizar tres colecciones principales para organizar los contenidos, las sugerencias y los administradores.**
- **Todavía necesito implementar y probar la conexión real con MongoDB y las rutas de Express.**
- **Necesito apoyo para asegurar la autenticación administrativa y conectar las funciones actuales del frontend con la base de datos.**
- **Dejaré las funciones avanzadas para después de terminar el MVP.**

La estructura de carpetas hacia la que vas

```
mi-proyecto/
├── backend/
│   ├── index.js ← Express: los endpoints
│   ├── models/ ← Mongoose: los modelos de la Parte 6 │   ├── .env ← la cadena de
```

conexión (NUNCA a GitHub) │ └── package.json
├── frontend/
│ ├── src/
│ │ ├── App.jsx
│ │ ├── componentes/ ← los componentes de la Parte 9
│ │ └── pantallas/ ← las pantallas de la Parte 9
│ └── package.json
├── arquitectura.md ← este taller
└── .gitignore ← node_modules y .env

AUTOEVALUACIÓN

Antes de entregar, revisa tu documento con esta lista. Si todo da , estás listo para construir.

#	Criterio	
1	Puedo explicar mi app en **una frase**, sin palabras técnicas	
2	Tengo **máximo 5** funcionalidades imprescindibles	
3	Escribí **al menos 5 historias de usuario** con su "para"	
4	Cada historia dice **cuándo está lista** (criterios de aceptación)	
5	Cada historia tiene **un endpoint** que la cumple	
6	Cada endpoint sabe **de qué colección** saca o guarda los datos	
7	Cada colección tiene sus campos con **tipo** y si es **obligatorio**	
8	Cada pantalla sabe **qué endpoint** le da los datos	
9	Pensé **qué se ve cuando no hay datos** en cada pantalla	
10	Decidí si necesito registro de usuarios **y por qué**	
11	Escribí **tres cosas que NO voy a hacer**	
12	Puedo recorrer mi historia principal **de punta a punta sin huecos**	

EJEMPLO COMPLETO RESUELTO

Para que veas cómo se ve un taller bien respondido. Caso: **Patitas Felices** (paseo de mascotas)

. 1. Qué es

> "Mi aplicación sirve para que los dueños de mascotas puedan reservar un paseo sin tener que llamar por teléfono."

Hoy la gente lo hace por WhatsApp, y el paseador pierde reservas porque no alcanza a contestar.

2. Qué hace

Imprescindible (MVP)
1. Ver los servicios disponibles

2. Ver el detalle de un servicio
3. Reservar un paseo
4. Ver mis reservas
5. (Admin) Crear y editar servicios

☐ **Deseable:** cancelar una reserva · calificar al paseador · filtrar por tamaño de mascota ☐ **Soñado:** chat con el paseador · seguimiento en el mapa · pagos en línea

☐ **Fuera de alcance:** pagos reales · app de celular · notificaciones por correo **Acción número uno:** reservar un paseo.

3. Historias de usuario

Historia 1 — Reservar un paseo
Como dueño de una mascota,
quiero reservar un paseo eligiendo el servicio y la fecha,
para no tener que llamar por teléfono y esperar respuesta.

Lista cuando:
- Puedo elegir un servicio de la lista
- Puedo elegir fecha y hora
- Si dejo un campo vacío, veo un mensaje claro y no se guarda nada
- Al confirmar veo "¡Reserva confirmada!"
- La reserva aparece después en "Mis reservas"

Historia 2 — Ver mis reservas
Como dueño de una mascota,
quiero ver todas mis reservas en un solo lugar,
para saber cuándo es el próximo paseo sin tener que buscar en el chat.

Lista cuando:
- Veo solo mis reservas, no las de otros
- Cada reserva muestra servicio, fecha y estado
- Si no tengo ninguna, veo "Aún no tienes reservas. ¡Agenda tu primer paseo!"

Historia 3 — Publicar un servicio
Como administrador,
quiero crear y editar los servicios que ofrecemos,
para cambiar los precios sin tener que pedirle ayuda a un programador.

Lista cuando:
- Puedo crear un servicio con nombre, precio y tamaño de mascota
- Si falta el precio o es negativo, veo un error y no se guarda
- Puedo editar un servicio existente
- Los cambios se ven de inmediato en el catálogo

4. Roles

Dueño	Ver servicios, reservar, ver sus reservas	Crear servicios ni ver reservas ajenas
Paseador	Ver las reservas asignadas a él	Crear servicios ni ver datos de otros paseadores
Admin	Crear, editar y borrar servicios; ver todas las reservas	—

¿Necesito registro? Sí, porque cada dueño debe ver **sus** reservas y nadie más.

5. Modelo de datos

Colección `servicios`

nombre	String	Sí	"Paseo 1 hora"
precio	Number	Sí (mínimo 0)	15000
tamanoMascota	String con opciones	Sí	"mediano"
descripcion	String	No	"Incluye agua y bolsas"
activo	Boolean	Sí (por defecto true)	true

Colección `usuarios`

nombre	String	Sí	"Ana Ríos"
email	String (único)	Sí	"ana@mail.com"
rol	String con opciones	Sí	"dueño"
telefono	String	No	"3001234567"
mascota	Object embebido	No	`{ nombre: "Lola", tamano: "mediano" }`

Colección `reservas`

emailCliente	String	Sí	"ana@mail.com"
servicio	String	Sí	"Paseo 1 hora"
precio	Number	Sí	15000
fecha	Date	Sí	new Date()
estado	String con opciones	Sí (por defecto "pendiente")	"pendiente"

Decisiones de diseño:

- La **mascota va embebida** en el usuario porque pertenece solo a él y siempre se consulta junto con él.
- El cliente de una reserva va como **referencia** (su email), porque el usuario existe aunque se borre la reserva.
- El **precio sí se guarda en la reserva**, aunque parezca que se puede calcular. Si mañana sube el precio del servicio, las reservas viejas deben conservar el precio del día en que se hicieron. *Lo que se calcula no se guarda... salvo cuando es una foto del pasado.*

6. Endpoints

Ver servicios	GET	/api/servicios	Lista	200
Ver un servicio	GET	/api/servicios/:id	Uno	200 / 404
Crear servicio (admin)	POST	/api/servicios	El creado	201 / 400
Editar servicio (admin)	PUT	/api/servicios/:id	El editado	200 / 400 / 404
Reservar	POST	/api/reservas	La reserva	201 / 400
Ver mis reservas	GET	/api/reservas?email=...	Lista	200

7. Pantallas y componentes

Inicio	Todos	—	—
Catálogo	Todos	GET /api/servicios	"Pronto tendremos servicios disponibles"
Detalle + reservar	Dueño	GET /api/servicios/:id + POST /api/reservas	—
Mis reservas	Dueño	GET /api/reservas?email=...	"Aún no tienes reservas. ¡Agenda tu primer paseo!"
Panel admin	Admin	GET POST PUT /api/servicios	"No has creado servicios todavía"

Componentes que se repiten: Encabezado · PiePagina · TarjetaServicio · MensajeVacio · BotonPrincipal

8. Recorrido de la historia principal

El usuario hace clic en "Reservar"
↓
React ejecuta fetch("/api/reservas", { method: "POST" }) con los datos
↓
Llega al endpoint POST /api/reservas
↓
Express valida: que venga email, servicio y fecha; que la fecha no sea pasada
↓
Mongoose / MongoDB ejecuta: Reserva.create({ ... })
↓
Responde con el código 201 y la reserva creada
↓
React muestra "¡Reserva confirmada para el 12 de marzo!"

9. Decisiones y pendientes

- **Decidí** no hacer pagos en línea, porque quiero alcanzar a terminar el MVP.
-

Todavía no sé cómo proteger las contraseñas → lo pregunto en mentoría. •
Todavía no sé cómo saber qué usuario está conectado → lo veremos en autenticación. •
Necesito apoyo en cómo desplegar el proyecto para que otras personas lo puedan usar.

SI TE TRABAS

No se me ocurren 10 acciones	Abre una app parecida a la tuya y escribe todo lo que puedes hacer en ella
Todas mis funcionalidades me parecen imprescindibles	Pregúntate: *"si el semestre se acabara en dos semanas, ¿cuál muestro?"*
No sé si algo va en una colección o dentro de otra	Pregúntate: *"¿esto existe por su cuenta, o solo tiene sentido dentro de lo otro?"*
No sé qué verbo usar en un endpoint	¿Trae datos? GET. ¿Crea? POST. ¿Modifica? PUT. ¿Borra? DELETE
Me trabo en el mapa de la Parte 10	**Perfecto: eso es justo lo que vas a preguntar.** Escríbelo tal cual y llévalo a la próxima sesión
Siento que mi idea no sirve	Habla con tu mentor antes de cambiarla. Casi siempre no es la idea: es que falta recortarla

Lo último, y lo más importante:
El código no es lo difícil. Lo difícil es decidir qué construir. Lo que acabas de escribir vale más que cien líneas de código, porque es lo que te dice cuáles cien líneas escribir.